

CURSO TÉCNICO EM INFORMÁTICA

Linguagem de Programação Estruturada

Aula 1 — História e Fundamentos da Programação

Professor: Rafael de Olanda · Turma: TI1F



Agenda da aula 1

- Boas-vindas e apresentação da disciplina
- 1. História da programação
- 2. O que é programação estruturada
- 3. Abstração e algoritmo
- 4. Baixo nível vs alto nível
- 5. Compiladores vs interpretadores
- 6. Linguagens de programação e mercado
- Recap e encerramento

Sobre a disciplina

- **Linguagem de Programação Estruturada**
 - Foco principal: linguagem C · paralelo com Pascal (e Delphi/Lazarus? – plano de trabalho)
- **Pré-requisito**
 - Lógica de Programação
- **Como serão as avaliações**
 - 2 avaliações teóricas + 1 avaliação prática
 - Ao final do curso vocês terão condições de criar um programa (simples) completo.



1

História da Programação

Como chegamos das primeiras máquinas até C e Pascal

Linha do tempo da programação

- Anos 1940** [Cartões perfurados](#) e linguagem de máquina — programar era literalmente ligar fios e furar cartão.
- Anos 1950** Assembly (mais legível que binário) e Fortran, uma das primeiras linguagens de alto nível.
- 1970** Pascal, criado por Niklaus Wirth especificamente para ensinar programação estruturada.
- 1972** Linguagem C, criada por Dennis Ritchie nos Bell Labs — **até hoje base de sistemas operacionais.**
- 1990–2000** Java, PHP, JavaScript — a explosão da internet e da programação orientada a objetos.
- Hoje** Python, linguagens modernas e assistentes de IA que ajudam (mas não substituem) quem entende lógica.

Por que aprender C e Pascal em 2026?

- **C ainda roda o mundo**
 - Sistemas operacionais (Linux, partes do Windows), drivers, sistemas embarcados, e é a base de que várias linguagens modernas (Python, PHP) são implementadas
- **Pascal nasceu para ensinar**
 - Sintaxe clara e verbosa (begin/end, :=) ajuda a fixar a lógica antes de partir para sintaxes mais compactas como a de C
- **Fundamento é o que dura**
 - Quem entende como o computador realmente processa um programa aprende qualquer linguagem nova depois — inclusive as que a IA sugere ou gera hoje em dia



2

O que é Programação Estruturada

O nome da disciplina, explicado de verdade

O que é programação?

*A programação é o processo de dar instruções a um computador para que ele resolva problemas ou execute tarefas específicas por meio de algoritmos. (Paradigma da programação **imperativa**)*

1

Entrada (Input)

São os dados que o programa recebe do mundo exterior.

2

Processamento

Fase que o programa manipula os dados que recebeu, seguindo as regras do algoritmo.

3

Saída (Output)

São os dados que o programa apresenta após processar as informações.

Evolução da programação

1 LINGUAGEM LINEAR DE BAIXO NÍVEL

Execução sequência de instruções



Características

- Controle total do hardware
- Instruções passo a passo
- Difícil de entender e manter
- Específica da arquitetura

Exemplo (estilo Assembly - GO TO)

```
INICIO: LOAD R1, 0      ; R1 = 0
        LOAD R2, 10     ; R2 = 10
LOOP:   ADD R1, 1        ; R1 = R1 + 1
        CMP R1, R2      ; compara R1 e R2
        JLT LOOP        ; GO TO LOOP se R1 < R2
        HALT            ; fim do programa
```

Fluxo de controle com saltos (GO TO) e manipulação direta de registradores.

2 PROGRAMAÇÃO ESTRUTURADA

Organização com sequência, seleção e repetição



Características

- Usa estruturas de controle
- Código mais claro e legível
- Facilita testes e manutenção
- Independente do hardware

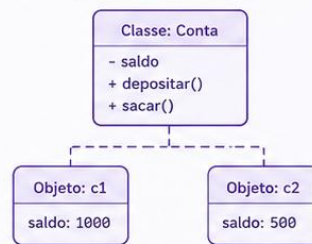
Exemplo (Pseudocódigo Estruturado)

```
início
    contador ← 0
    limite ← 10
    enquanto contador < limite faça
        contador ← contador + 1
    fim_enquanto
fim
```

Fluxo de controle estruturado: sequência, seleção (se) e repetição (enquanto).

3 PROGRAMAÇÃO ORIENTADA A OBJETOS

Organização em objetos que encapsulam dados e comportamentos



Características

- Baseada em classes e objetos
- Encapsulamento, herança e polimorfismo
- Reuso de código
- Modela o mundo real

Exemplo (Código Orientado a Objetos - Python)

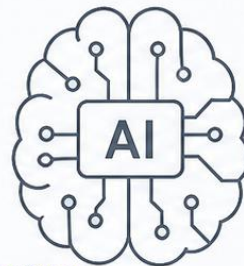
```
class Conta:
    def __init__(self, saldo):
        self.saldo = saldo
    def depositar(self, valor):
        self.saldo += valor
    def sacar(self, valor):
        self.saldo -= valor

c1 = Conta(1000)
c1.depositar(200)
```

Dados (saldo) e comportamentos (métodos) agrupados em objetos.

4 INTELIGÊNCIA ARTIFICIAL

Sistemas que aprendem, raciocinam e tomam decisões



Características

- Aprende com dados
- Identifica padrões
- Toma decisões inteligentes
- Melhora continuamente

Exemplo (IA - Aprendizado de Máquina)

```
# Treinar modelo com dados
modelo.treinar(dados_rotulados)

# Fazer previsões
resultado = modelo.prever(novos_dados)

# Exemplo: classificar imagens, recomendar,
detectar fraudes, etc.
```

Modelos aprendem com dados e geram decisões ou previsões de forma autônoma.

Programação estruturada

Teorema de Böhm–Jacopini (1966): qualquer algoritmo pode ser escrito usando só estas 3 estruturas de controle — sem "saltos" desordenados pelo Código (Forma primitiva com saltos "GOTO" = difícil de manter)

1

Sequência

As instruções são executadas uma após a outra, na ordem em que foram escritas.

2

Decisão (seleção)

O programa escolhe um caminho ou outro, dependendo de uma condição (if/else).

3

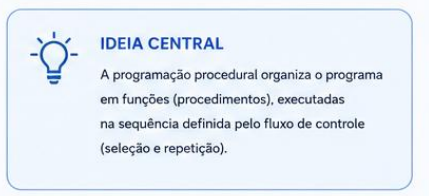
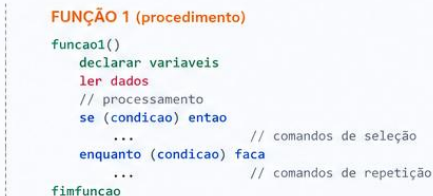
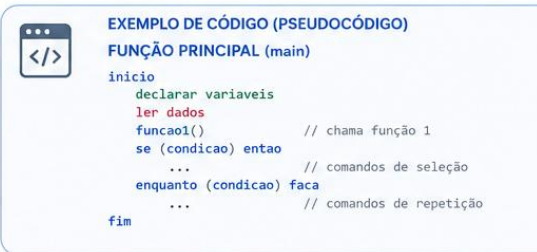
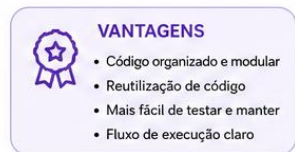
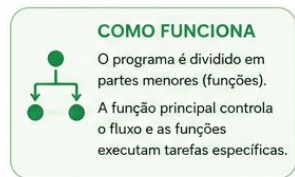
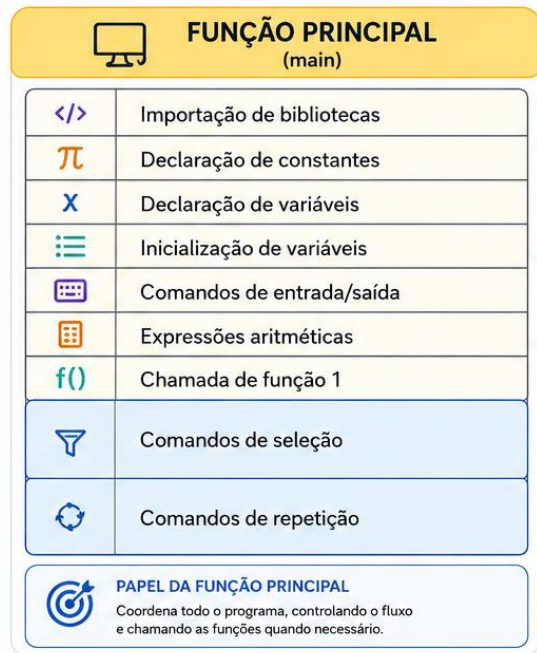
Repetição

Um bloco de instruções se repete enquanto uma condição for verdadeira (laços).

Edsger Dijkstra formalizou esse movimento em 1968, no artigo "Go To Statement Considered Harmful" — o manifesto que popularizou a programação estruturada

PROGRAMAÇÃO ESTRUTURADA (PROCEDURAL)

Decomposição do problema em funções (procedimentos)



3

Abstração e Algoritmo

Da ideia no papel até o computador entender o que fazer

O que é abstração?

- **Definição:** esconder os detalhes complexos e mostrar só o que importa para quem está usando
- **Exemplo do dia a dia**
 - Ao dirigir um carro, você usa o volante e o pedal — não precisa saber como o motor queima combustível para fazer as rodas girarem
- **Em programação**
 - Quando escrevemos $\text{soma} = a + b$, não pensamos em como o processador representa esses números em binário e os soma no nível elétrico
 - Cada linguagem de programação é uma camada de abstração sobre a anterior — é justamente isso que veremos no próximo bloco (baixo nível vs alto nível)

O que é um algoritmo?

- **Definição:** uma sequência finita e ordenada de passos para resolver um problema
- **Exemplo cotidiano — trocar um pneu furado:**
 - 1. Parar o carro em local seguro e sinalizar
 - 2. Pegar o macaco, a chave de roda e o step
 - 3. Afrouxar os parafusos da roda furada
 - 4. Levantar o carro com o macaco
 - 5. Trocar a roda furada pelo step e apertar os parafusos
- Isso é um algoritmo — mesmo sem nenhuma linha de código

Mesma lógica, formas diferentes de representar

Problema: somar dois números digitados pelo usuário

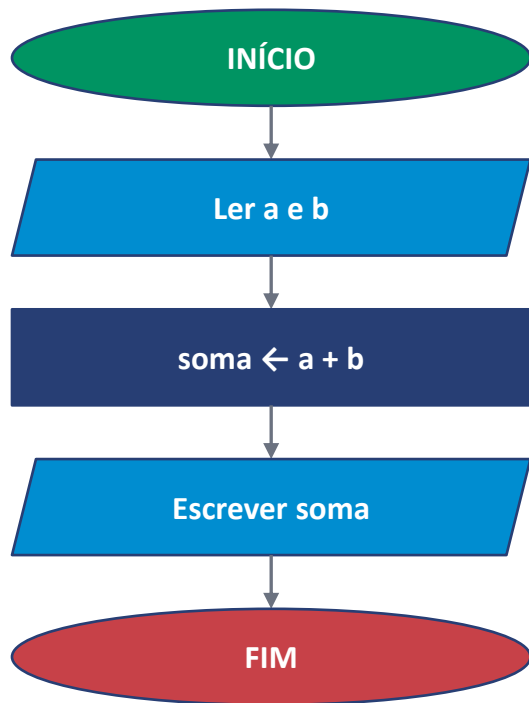
1. Algoritmo (linguagem natural)

1. Ler o primeiro número (a)
2. Ler o segundo número (b)
3. Somar a com b
4. Mostrar o resultado

2. Pseudocódigo

```
INÍCIO  
  LER a  
  LER b  
  soma ← a + b  
  ESCREVER soma  
FIM
```

3. O mesmo algoritmo em fluxograma



Símbolos

- Oval = início/fim
- Paralelogramo = entrada/saída
- Retângulo = processamento

4. E em código real — C e Pascal

Isso vocês vão escrever de verdade a partir da aula 5 — o mesmo algoritmo, nas duas linguagens

C

```
#include <stdio.h>

int main() {
    int a, b, soma;

    scanf("%d %d", &a, &b);
    soma = a + b;
    printf("%d", soma);

    return 0;
}
```

Pascal

```
program Soma;
var
    a, b, soma: integer;

begin
    readln(a, b);
    soma := a + b;
    writeln(soma);
end.
```



4

Baixo Nível vs Alto Nível

Quanto controle sobre a máquina eu realmente preciso ter?

Baixo nível vs alto nível

	Baixo nível (Assembly/binário)	Alto nível (C, Python, JS...)
Controle sobre o hardware	Total	Abstraído pela linguagem
Velocidade de desenvolvimento	Lenta	Rápida
Portabilidade	Baixa (específico da CPU)	Alta (mesmo código, várias plataformas)
Curva de aprendizado	Íngreme	Mais suave
Exemplo do mesmo cálculo	MOV AX,5 \ ADD AX,BX	soma = a + b

Baixo nível na prática — Compiler Explorer

- **Demonstração ao vivo em godbolt.org**
 - Vamos escrever o mesmo soma = a + b em C e ver, em tempo real, para que instruções de baixo nível (assembly) isso é traduzido pelo compilador
- **O que observar**
 - Cada linha de C "alto nível" vira várias instruções de máquina
 - Coisas que parecem simples (soma, print) escondem bastante trabalho por baixo
 - Isso é abstração acontecendo na prática, ao vivo, na tela

Vídeo — programando (de verdade) um jogo de SNES



"Eu tentei programar um jogo no SNES... e não era nada do que eu esperava"

youtube.com/watch?v=dcVdn20ks_U

5

Compiladores vs Interpretadores

Duas formas diferentes de transformar código em execução

Como um compilador entende seu código



O interpretador faz basicamente as mesmas 3 primeiras etapas — só que a cada execução, e sem gerar um executável separado no final

Compilado vs interpretado

Compilado

Código-fonte → compilador → executável (código de máquina) → roda direto no processador

- Precisa recompilar a cada mudança
- Tende a rodar mais rápido
- **Exemplos: C, Pascal, C++, Rust**

Interpretado

Código-fonte → interpretador lê e executa linha a linha, em tempo de execução

- Não precisa de um passo de compilação separado
- Mais flexível para testar rápido
- **Exemplos: Python, JavaScript, PHP**



6

Linguagens e o Mercado Hoje

Onde cada linguagem aparece no dia a dia de quem programa

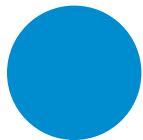
Exemplos de linguagens e onde se usam hoje

Linguagem	Nível	Tipo	Uso típico hoje
C	Médio/baixo	Compilada	Sistemas operacionais, drivers, sistemas embarcados
Pascal	Alto	Compilada	Ensino de estruturada, sistemas legados
Python	Alto	Interpretada	Automação, dados, Inteligência Artificial
Java	Alto	Compilada p/ bytecode + JVM	Sistemas corporativos, apps Android
JavaScript	Alto	Interpretada/JIT	Web — frontend e, com Node, backend também
PHP	Alto	Interpretada	Backend de sites, CMS (ex.: WordPress)

Pascal em destaque

- **Criado em 1970 por Niklaus Wirth, na ETH Zurique**
 - Batizado em homenagem ao matemático Blaise Pascal
- **Criado com um propósito pedagógico explícito**
 - Ensinar boas práticas de programação estruturada de forma clara e disciplinada
- **Sintaxe verbosa por design**
 - begin / end no lugar de chaves, := para atribuição — mais "lento de digitar", mas difícil de ler errado
- **Legado**
 - Influenciou o Delphi e ainda aparece em ensino de programação e sistemas legados

Panorama rápido do mercado de programação



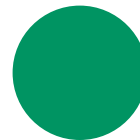
Frontend

A parte do sistema com que o usuário interage diretamente — telas, botões, formulários.



Backend

A lógica e os dados que ficam "por trás" — processa regras de negócio, acessa banco de dados.



API

A forma padronizada como dois sistemas diferentes conversam entre si e trocam informação.

Vamos aprofundar isso (com CRM, ERP, PDV e o papel da IA) na aula 4 — por enquanto, fica o gancho: todo esse ecossistema começa com a lógica que vocês vão treinar aqui.

Recapitulando a aula 1

- Programação estruturada: sequência, decisão e repetição — sem desvios desordenados
- Procedural (funções) vs Orientada a Objetos (objetos) — dois times diferentes
- Algoritmo: sequência finita e ordenada de passos para resolver um problema
- Abstração: esconder detalhes complexos e mostrar só o que importa
- Do algoritmo ao código: algoritmo → pseudocódigo → fluxograma → linguagem real (C e Pascal)
- Baixo nível dá controle total; alto nível dá produtividade e portabilidade
- Compilador: léxica → sintática → semântica → código de máquina
- Compilado (C, Pascal) vs interpretado (Python, JS, PHP)

Próxima aula

- **Aula 2 — Sintaxe, semântica e dados**
- Erro de sintaxe vs erro de semântica
- Tipos de dados em C (int, float, char...)
- Variáveis, constantes
- Arrays/vetores — o conceito, antes da sintaxe

Obrigado!



Dúvidas? Vamos conversar antes da próxima aula.